

Tier 3: Test Framework Architecture

Assignment 9: PyTest Integration with HTML Reporting

1. Problem Statement: -

Convert your setup to run via PyTest. Use PyTest fixtures for driver initialization and teardown. Run your test suite from the terminal and configure it to auto-generate an HTML execution report with embedded screenshots of any failed test steps.

2. Objective: -

To integrate the PyTest framework for test execution, fixture management, and detailed HTML report generation.

3. Tools, Software, and Concepts Used: -

Python 3.x, Selenium WebDriver, PyTest, pytest-html.

4. Source Code / Implementation: -

```
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By
@pytest.fixture(scope="class")
def setup(request):
    driver = webdriver.Chrome()
    driver.implicitly_wait(10)
    request.cls.driver = driver
    yield
    driver.quit()
@pytest.mark.usefixtures("setup")
class TestLogin:
    def test_valid_login(self):
        self.driver.get("https://www.saucedemo.com/")
        self.driver.find_element(By.ID, "user-
name").send_keys("standard_user")
        self.driver.find_element(By.ID,
"password").send_keys("secret_sauce")
        self.driver.find_element(By.ID, "login-button").click()
        assert "/inventory.html" in self.driver.current_url
    def test_invalid_login_failure(self):
        self.driver.get("https://www.saucedemo.com/")
        self.driver.find_element(By.ID, "user-
name").send_keys("locked_out_user")
        self.driver.find_element(By.ID,
"password").send_keys("secret_sauce")
```

```

self.driver.find_element(By.ID, "login-button").click()
# Deliberately asserting wrong text to trigger a failure and
screenshot in HTML report
error_msg = self.driver.find_element(By.CSS_SELECTOR, "h3[data-
test='error']").text
assert "Welcome" in error_msg, "Error message did not match!"

```

5. Output & Screenshots: -

report.html
Report generated on 23-Sep-2026 at 15:42:32 by [pytest-html](#) v4.2.0

Environment

Python	3.10.9
Platform	Windows-10-10.0.22000-SP0
Packages	• pytest: 9.1.1 • pluggy: 1.6.0
Plugins	• anyio: 4.13.0 • HTML: 4.2.0 • metadata: 3.1.1

Summary
2 tests took 00:00:05

(Un)check the boxes to filter the results.

1 Failed, 1 Passed, 0 Skipped, 0 Expected failures, 0 Unexpected passes, 0 Errors, 0 Reruns, 0 Retried

Result	Test	Duration	Links
Failed	Ass9.py:TestLogin: test_invalid_login_failure	00:00:02	
Passed	Ass9.py:TestLogin: test_valid_login	00:00:03	

Failed Test Details:

```

self = <Ass9.TestLogin object at 0x000001749ACB8700>

def test_invalid_login_failure(self):
    self.driver.get("https://www.saucedemo.com/")
    self.driver.find_element(By.ID, "user-name").send_keys("locked_out_user")
    self.driver.find_element(By.ID, "password").send_keys("secret_sauce")
    self.driver.find_element(By.ID, "login-button").click()
    # Deliberately asserting wrong text to trigger a failure and screenshot in HTML report
    error_msg = self.driver.find_element(By.CSS_SELECTOR, "h3[data-test='error']").text
    assert "Welcome" in error_msg, "Error message did not match!"
>
E   AssertionError: Error message did not match!
E   assert 'Welcome' in 'Epic sadface: Sorry, this user has been locked out.'
Ass9.py:33: AssertionError

```

Fig.1: Successfully generated report.html using the pytest-html plugin.

6. Result, Observation, and Conclusion: -

- **Result:** Tests executed successfully via PyTest, leveraging fixtures for clean setup/teardown. A comprehensive HTML report was generated.
- **Observation:** PyTest fixtures streamline initialization processes, and HTML reports provide clear, easily shareable visibility into test execution health and failure points.
- **Conclusion:** Utilizing a standardized test runner and reporting tool is essential for managing enterprise-level automation suites.